

Q. Consider the solution of Bounded-buffer producer-consumer problem using semaphores:

shared data:

Semaphore mutex=1, empty=N, full=0;

producer process:

```
do{
    //produce an item
    wait(empty);
    wait(mutex);
    buffer...
    //add the item into buffer...
    signal(mutex);
    signal(full);
    item...
}while(true);
```

consumer process:

```
do{
    wait(full);
    wait(mutex);
    //remove an item from
    signal(mutex);
    signal(empty);
    //consume the removed
}while(true);
```

(a) Suppose that the order of the instructions

```
wait(empty);
wait(mutex);
```

in producer process is changed to,

```
wait(mutex);
wait(empty);
```

Will the processes run in the same way as before? Comment on a possible problem otherwise.

Suppose that all buffers are full and producer process executes wait(mutex) instruction, decreasing mutex to 0. Then, producer will wait due to empty=0 and consumer will wait due to mutex=0 for ever. Hence, they will be deadlocked.

Q. Consider the following proposed solution to the dining philosophers problem that we have discussed in the class:

Shared data : semaphore chopstick[5]={ 1, 1, 1, 1, 1 };
 semaphore number_of_philosophers= 4 ;

Philosopher i:

do {

 think();

wait(number_of_philosophers);_____

 wait(chopstick[i]);

 wait(chopstick[(i+1) % 5]);

 ...

eat

 ...

 signal(chopstick[i]);

 signal(chopstick[(i+1) % 5]);

signal(number_of_philosophers);_____

 ...

 } while (true);

Fill in the empty spaces in a way that, the given solution will have no deadlock and then explain your reasons below.

number_of_philosophers =4 means we may have maximum 4 processes in the system, so that one of them can finish and others can continue without deadlock.

Q. Suppose that N processes, $P_0, \dots, P_{(N-1)}$ need to share some critical region of code (critical section), which only one at a time is allowed to execute. These processes are synchronized as given below:

Shared Data

```
int turn = 0;
```

Process P_i

```
while( true )
{
    while( turn != i );

    Critical Section;

    turn = (turn + 1) % N;

    Remainder Section;
}
```

a. Is Mutual Exclusion satisfied? Why or why not? Explain.

Yes. When a process P_i wants to enter its CS, if $turn$ is different than i , it has to wait. Hence, when $turn=k$, only process p_k is allowed to enter.

b. Is progress satisfied? Why or why not? Explain.

Progress is not satisfied. When a process P_i is running in its remainder section, the value of $turn$ may be equal to i and none of the other processes are allowed to enter their CS's until P_i enters once.

Q. Assume that there are two concurrent processes as given below. The shared data are:

Semaphore R=0, S=1;

Process A:

```
Wait(R);  
Wait(S);  
Instruction A.1;  
Signal(S);  
Instruction A.2;  
Signal(R);
```

Process B:

```
Instruction B.1;  
Signal(R);  
Instruction B.2;  
Wait(S);  
Instruction B.3;  
Signal(S);
```

Determine whether the following statements are true (T) or false (F):

- a) **Instruction A.1** cannot start until **Instruction B.1** is complete.
(_ T _)
- b) **Instructions A.1** and **B.2** cannot execute simultaneously.
(_ F _)
- c) **Instructions A.1** and **B.3** cannot execute simultaneously
(_ T _)
- d) **Instructions A.2** and **B.3** cannot execute simultaneously
(_ F _)
- e) The system may deadlock.
(_ F _)

Q. For each program given below, indicate whether it could exhibit a deadlock, a race condition, or both. If so, explain why. Assume that the function process1 runs in one process and the function process2 runs in another process, and that the following data are shared between processes, and initialized as indicated prior to execution in each case.

Shared Data:

```
semaphore m = 1;  
semaphore n = 1;  
boolean lm = false;  
boolean ln = false;  
int x = 0;  
int y = 0;
```

(a)

```
process1() {  
    wait(m);  
    x++;  
    wait(n);  
    y++;  
    signal(n);  
    signal(m);  
}
```

```
process2() {  
    wait(n);  
    x++;  
    wait(m);  
    y++;  
    signal(m);  
    signal(n);  
}
```

Answer: Race condition and deadlock.



Process P0:

```
int count;  
for(count=0;count<=N;count++)  
    tally++;
```

Assume that two processes running this code share the following variables:

```
const int N = 10;  
int tally = 10;
```

(a) Determine the proper lower and upper bound on the final value of the shared variable **tally** after all the instructions are executed. Assume that a value can only be incremented after it has been loaded into a CPU register by a separate machine instruction.

upper value:32.....

lower value:21.....

(b) Suppose that an K of these processes are allowed to execute in parallel under the assumptions of part (a). What are the lower and upper bound on the final value of the shared variable **tally** in this case.

upper value:10+K*11.....

lower value:21.....

$$21 \leq \text{tally} \leq 10 + K * 11$$

K=1 for lower value

Q. The processes A, B and C are given below.

a. Using only **one** semaphore variable, synchronize the processes such that

- produceA is executed first,
- produceB **or** produceC may be the second.

ABC or ACB

Shared data:

Semaphore **s=0;**

Process A:

{

//produceA;

signal(s);
signal(s);

}

Process B:

{

wait(s);
//produceB;

}

Process C:

{

wait(s);
//produceC;

}

b. Using only **one** semaphore variable, synchronize the processes such that

- produceB or produceC may be executed the first,
- produceA is executed the last.

BCA or CBA

Shared data:

Semaphore **s=0;**

Process A:

{

wait(s);
wait(s);

//produceA;

}

Process B:

{

//produceB;

signal(s);

}

Process C:

{

//produceC;

signal(s);

}

Q. Add the semaphores necessary to synchronize processes A, B, C, and D so that A completes before any other process (B, C, or D) starts, and B completes before C or D may execute, but C and D may execute concurrently. You have to use minimum number of semaphore variables as possible.

semaphoreS1=0 S2=0.....;

ABCD or ABDC

Process A:	Process B:	Process C:	Process D:
{	{	{	{
- do work of A	wait (S1);	wait (S2);	wait (S2);
signal (S1);	- do work of B	- do work of C	- do work of D
}	signal (S2);	}	}
	signal (S2);		
	}		

b) (5 points) Suppose that a process is executing "counter=counter+1" while another process is executing **concurrently** (without any synchronization) "counter=counter-2", where the *counter* is a variable shared between the two processes and is only changed in those statements. Given that the value of *counter* is **five** before execution, give all the possible value(s) after both processes finish their statement.

Counter= 5+1=6 , 5-2=3, 5-2+1=4

counter=5

counter=counter+1

counter=counter-2

R1=counter=5

R1=R1+1

.

.

counter=R1

counter=3

counter=6

R2=counter=5

R2=R2-2

Counter=R2

Q. Write the following three functions employing semaphores: ProducePart1(), ProducePart2(), and Join(). ProducePart1() function produces a single unit of part1. Similarly, ProducePart2() produces a single unit of part2. Join () produces a machine using both parts if they are available. These processes are running concurrently and any part can be produced at any time, and these processes may run in any order.

Shared data:

S1,S2=0 S3,S4=1

ProducePart1, ProducePart2, Join or
ProducePart2, ProducePart1, Join

ProducePart1 ():	ProducePart2 ():	Join ():
{ <u>wait(s3);</u> - produce Part 1 <u>signal(s1);</u> }	{ <u>wait(s4);</u> - produce Part 2 <u>signal(s2);</u> }	{ <u>wait(s1);</u> <u>wait(s2);</u> - join parts <u>signal(s3);</u> <u>signal(s4);</u> }

run in
while
loop

Consider three concurrent processes A, B and C. Suppose a shared variable X with an initial value of 10 is increased by 2 steps by process A, increased by 3 steps by process B, and decreased by 1 step by process C. Each process accesses X only once. By providing code for the three processes, synchronize processes using semaphores (use minimum number of semaphores) such that decrement must be done after increments are performed. Increments can be in any order.

ABC or BAC

Possible outputs without synchronization:
12, 13, 14, ...